

OPOFIT

Documentacion Tecnica Completa

Aplicacion de Preparacion Fisica para Oposiciones

Backend • Frontend • Base de Datos

Autor: Rafael Javier Luengo Barreno

Proyecto Final de Grado

2026

INDICE DE CONTENIDOS

1. Introduccion General
 2. Arquitectura del Proyecto
 3. Base de Datos (MySQL)
 4. Backend (Node.js / Express)
 5. Frontend (Android / Kotlin / Jetpack Compose)
 6. Flujo de Autenticacion
 7. Flujos de Datos Principales
 8. Seguridad
 9. Testing
 10. Despliegue y Configuracion
-

1. INTRODUCCION GENERAL

1.1 Que es OpoFit

OpoFit es una aplicacion movil Android disenada para ayudar a los opositores a preparar las pruebas fisicas de las oposiciones a Cuerpos de Seguridad del Estado en Espana (Policia Nacional y Guardia Civil). La aplicacion proporciona:

- **Rutinas de entrenamiento personalizadas** segun el nivel del usuario (BASICO, INTERMEDIO, AVANZADO)
- **Baremos oficiales del BOE** para las pruebas fisicas de cada oposicion
- **Seguimiento de progreso** con historial de entrenamientos
- **Rutinas libres personalizadas** que el usuario puede crear a su gusto
- **Noticias y convocatorias** de cada oposicion
- **Calculo automatico de nivel** basado en las marcas del usuario
- **Soporte para ambos generos** con baremos diferenciados

1.2 Stack Tecnologico

Componente	Tecnologia	Version
Frontend	Android (Kotlin) + Jetpack Compose	SDK 24-36
Backend	Node.js + Express	Express 5.2.1
Base de Datos	MySQL	5.7+
Autenticacion	JWT + Google Sign-In + Firebase Auth	-
API REST	Retrofit (frontend) / Express Routes (backend)	Retrofit 2.9.0
Almacenamiento local	DataStore Preferences	1.0.0

1.3 Estructura del Repositorio

```
Proyecto_FG_Rafael_Javier_Luengo_Barreno/  
+-- BBDD/                                # Scripts de base de datos  
|   +-- mydb.sql                         # Esquema completo (15 tablas)  
|   +-- seed.sql                        # Datos iniciales reales del BOE  
|   +-- diagrama_relacional.mwb         # Diagrama ER (MySQL Workbench)  
+-- backend/                            # API REST con Node.js/Express  
|   +-- app.js                          # Punto de entrada del servidor  
|   +-- src/                            # Codigo fuente organizado en capas  
|   +-- tests/                          # Tests unitarios con Jest  
+-- frontend/                           # App Android con Kotlin/Compose  
|   +-- app/src/main/java/              # Codigo fuente organizado en MVVM  
+-- Documentacion/                     # Documentacion del proyecto
```

2. ARQUITECTURA DEL PROYECTO

2.1 Arquitectura General

La aplicacion sigue una arquitectura cliente-servidor de tres capas:

```
FRONTEND (Android App) - Kotlin + Jetpack Compose + MVVM  
  Screens --> ViewModels --> Retrofit --> HTTP API  
      |  
      | HTTPS / JSON  
      v  
BACKEND (Node.js / Express) - API REST + JWT + Rate Limiting  
  Routes --> Middleware --> Controllers --> Services  
      |  
      | mysql2/promise  
      v  
BASE DE DATOS (MySQL) - 15 Tablas + InnoDB + Foreign Keys  
  Oposiciones, Usuarios, Rutinas, Historial, etc.
```

2.2 Patron del Backend: MVC por Capas

El backend sigue el patron Modelo-Vista-Controlador adaptado a una API REST:

- **Routes (Rutas):** Definen los endpoints HTTP y asignan el middleware y controlador correspondiente.
- **Middleware:** Valida tokens JWT antes de permitir el acceso a rutas protegidas.
- **Controllers (Controladores):** Validan la peticion, comprueban autorizacion y llaman a los servicios.
- **Services (Servicios):** Contienen toda la logica de negocio y las consultas a la base de datos.
- **Models (Modelos):** Clases JavaScript que documentan la estructura de los datos.

2.3 Patron del Frontend: MVVM

El frontend sigue el patron Model-View-ViewModel:

- **View (Pantallas):** Composables de Jetpack Compose que renderizan la interfaz.

- **ViewModel:** Gestiona el estado de la UI y realiza las llamadas a la API.
- **Model (Datos):** Data classes para peticiones, respuestas y almacenamiento local.

3. BASE DE DATOS (MySQL)

3.1 Configuración General

- **Nombre del esquema:** mydb
- **Motor:** InnoDB (todas las tablas)
- **Juego de caracteres:** UTF-8
- **Numero total de tablas:** 15

3.2 Diagrama Entidad-Relación

Las 15 tablas se organizan en 6 subsistemas funcionales:

Subsistema de Oposiciones: oposiciones, pruebas_oficiales, baremos_puntuacion, requisitos_nivel, noticias

Subsistema de Usuarios: usuarios, settings

Subsistema de Marcas: marcas_perfil

Subsistema de Rutinas Oficiales: rutinas_opo, detalle_rutina_opo, ejercicios

Subsistema de Rutinas Personales: rutinas_pers, detalle_rutina_pers

Subsistema de Historial: historial_sesiones, registro_resultados

3.3 Descripción Detallada de Cada Tabla

3.3.1 Tabla oposiciones

Propósito: Almacena los tipos de oposiciones disponibles.

Columna	Tipo	Restricciones	Descripción
id_oposicion	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
nombre	VARCHAR(200)	NOT NULL	Nombre de la oposicion

Datos semilla: 2 registros: Policía Nacional - Escala Basica (ID 1) y Guardia Civil - Acceso Libre (ID 2).

3.3.2 Tabla pruebas_oficiales

Propósito: Define las pruebas físicas oficiales de cada oposición.

Columna	Tipo	Restricciones	Descripcion
id_pruebas_oficiales	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
nombre_prueba	VARCHAR(200)	NOT NULL	Nombre de la prueba
descripcion	TEXT	NOT NULL	Descripcion detallada
trucos	TEXT	NULL (opcional)	Consejos y trucos
oposiciones_id_oposicion	INT	FK -> oposiciones, NOT NULL	Oposicion a la que pertenece
mejor_si_es_menor	TINYINT(1)	NOT NULL, DEFAULT 0	1=menor es mejor (tiempos), 0=mayor es mejor (repeticiones)

Datos semilla: 7 pruebas oficiales: - **Policia Nacional:** Circuito de agilidad, Dominadas/Suspension en barra, Carrera 1.000 metros - **Guardia Civil:** Carrera 2.000 metros, Circuito de agilidad, Flexiones de brazos, Natacion 50 metros

3.3.3 Tabla usuarios

Proposito: Almacena la informacion de los usuarios registrados.

Columna	Tipo	Restricciones	Descripcion
id_usuario	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
nombre	VARCHAR(100)	NOT NULL	Nombre del usuario
email	VARCHAR(100)	NOT NULL, UNIQUE	Email (unico, para login)
password	VARCHAR(255)	NOT NULL	Contraseña hasheada (bcrypt)
genero	ENUM('MUJER','HOMBRE')	NOT NULL	Genero del usuario
peso	DECIMAL(10,2)	NOT NULL	Peso en kg
altura	DECIMAL(10,2)	NOT NULL	Altura en metros
imc	DECIMAL(10,2)	NULL	Indice de Masa Corporal (calculado)
fecha_registro	DATETIME	NOT NULL	Fecha de registro
oposiciones_id_oposicion	INT	FK -> oposiciones, NULL	Oposicion objetivo

Formula del IMC: $\text{peso} / (\text{altura}/100)^2$

3.3.4 Tabla settings

Proposito: Almacena las preferencias del usuario (unidades de medida).

Columna	Tipo	Restricciones	Descripcion
id_setting	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
unidad_peso	VARCHAR(20)	NOT NULL	Unidad de peso: kg o lb
unidad_distancia	VARCHAR(20)	NOT NULL	Unidad de distancia: km o mi
usuarios_id_usuario	INT	FK -> usuarios, NOT NULL	Usuario propietario

Relacion: 1:1 con usuarios (cada usuario tiene una configuracion).

3.3.5 Tabla marcas_perfil

Proposito: Almacena los records personales del usuario en las pruebas oficiales.

Columna	Tipo	Restricciones	Descripcion
id_marcas_perfil	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
valord_record	DECIMAL(10,2)	NOT NULL	Valor del record
fecha_logro	DATETIME	NOT NULL	Fecha en que se logro
pruebas_oficiales_id_pruebas_oficiales	INT	FK -> pruebas_oficiales, NOT NULL	Prueba oficial
usuarios_id_usuario	INT	FK -> usuarios, NOT NULL	Usuario propietario

3.3.6 Tabla baremos_puntuacion

Proposito: Contiene los baremos oficiales del BOE para puntuar cada prueba. Escala de 0 a 10 puntos.

Columna	Tipo	Restricciones	Descripcion
id_baremo	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
pruebas_oficiales_id_pruebas_oficiales	INT	FK -> pruebas_oficiales, NOT NULL	Prueba oficial
genero	ENUM('HOMBRE','MUJER')	NOT NULL	Genero
marca_valor	DECIMAL(10,2)	NOT NULL	Valor de rendimiento
nota	INT	NOT NULL	Puntuacion (0-10)

Datos semilla: 140 registros (7 pruebas x 2 generos x 10 niveles de nota).

Ejemplo - Circuito de agilidad Policia Nacional (Hombres):

Tiempo (seg)	Puntos
11.70	0
11.40	1
10.80	3
10.20	5
9.40	7
8.80	8
8.60	9
8.20	10

3.3.7 Tabla requisitos_nivel

Proposito: Define los valores objetivo para cada nivel de entrenamiento.

Columna	Tipo	Restricciones	Descripcion
id_requisito_nivel	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
genero	ENUM('HOMBRE','MUJER')	NOT NULL	Genero
nivel_exigencia	INT	NOT NULL	1=BASICO, 2=INTERMEDIO, 3=AVANZADO
valor_objetivo	DECIMAL(10,2)	NOT NULL	Valor a alcanzar
pruebas_oficiales_id_pruebas_oficiales	INT	FK -> pruebas_oficiales, NOT NULL	Prueba

Datos semilla: 48 registros.

3.3.8 Tabla ejercicios

Proposito: Catalogo de ejercicios de entrenamiento disponibles.

Columna	Tipo	Restricciones	Descripcion
id_ejercicio	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
nombre	VARCHAR(200)	NOT NULL	Nombre del ejercicio
video_url	VARCHAR(255)	NULL	URL del video demostrativo
instrucciones_tecnicas	TEXT	NULL	Instrucciones tecnicas

Datos semilla: 20 ejercicios: Dominadas estrictas, Dominadas asistidas, Carrera continua 30 min, Series 400m, Series 200m, Flexiones de brazos, Circuito de conos, Natacion crol 50m, Series natacion 25m, Suspension en barra, Plancha abdominal, Sentadillas, Burpees, Zancadas, Fartlek 20 min, Remo invertido, Escalera de agilidad, Carrera continua 45 min, Press banca, Fondos en paralelas.

3.3.9 Tabla rutinas_opo

Proposito: Rutinas de entrenamiento prediseñadas para cada oposicion.

Columna	Tipo	Restricciones	Descripcion
id_rutina_opo	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
nivel	ENUM('BASICO','INTERMEDIO','AVANZADO')	NOT NULL	Nivel de dificultad
genero	ENUM('HOMBRE','MUJER')	NOT NULL	Genero
enfoque_tipo	ENUM('FUERZA','VELOCIDAD','RESISTENCIA')	NOT NULL	Enfoque del entrenamiento
oposiciones_id_oposicion	INT	FK -> oposiciones, NOT NULL	Oposicion

Datos semilla: 12 rutinas (2 oposiciones x 3 niveles x 2 generos).

Estructura de niveles: - BASICO + RESISTENCIA: Para usuarios principiantes - INTERMEDIO + FUERZA: Para usuarios con base - AVANZADO + VELOCIDAD: Para usuarios avanzados

3.3.10 Tabla detalle_rutina_opo

Proposito: Relaciona ejercicios con rutinas oficiales con sus parametros.

Columna	Tipo	Restricciones	Descripcion
id_detalle_rutina_opo	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
ejercicios_id_ejercicio	INT	FK -> ejercicios, NOT NULL	Ejercicio
rutinas_opo_id_rutina_opo	INT	FK -> rutinas_opo, NOT NULL	Rutina
repeticiones	INT	NOT NULL	Repeticiones por serie
series	INT	NOT NULL	Numero de series
descanso	INT	NOT NULL	Descanso entre series (segundos)

Datos semilla: 60 registros (12 rutinas x 5 ejercicios por rutina).

3.3.11 Tabla rutinas_pers

Proposito: Rutinas personalizadas creadas por los usuarios.

Columna	Tipo	Restricciones	Descripcion
id_rutina_pers	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
nombre_personalizado	VARCHAR(200)	NOT NULL	Nombre elegido por el usuario
usuarios_id_usuario	INT	FK -> usuarios, NOT NULL	Usuario creador

3.3.12 Tabla detalle_rutina_pers

Proposito: Ejercicios de las rutinas personales.

Columna	Tipo	Restricciones	Descripcion
id_detalle_rutina_pers	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
rutinas_pers_id_rutina_pers	INT	FK -> rutinas_pers, NOT NULL	Rutina personal
ejercicios_id_ejercicio	INT	FK -> ejercicios, NOT NULL	Ejercicio
series	INT	NOT NULL	Numero de series
repeticiones	INT	NOT NULL	Repeticiones por serie
descanso	INT	NOT NULL, DEFAULT 60	Descanso (segundos)

3.3.13 Tabla historial_sesiones

Proposito: Registra cada sesion de entrenamiento completada.

Columna	Tipo	Restricciones	Descripcion
id_historial_sesion	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
fecha_entreno	DATETIME	NOT NULL	Fecha y hora
tipo_rutina	ENUM('OPO','PERS')	NOT NULL	Tipo de rutina
duracion_oficial	INT	NOT NULL	Duracion en minutos
rutinas_pers_id_rutina_pers	INT	FK -> rutinas_pers, NULL	Rutina personal (si tipo=PERS)
rutinas_opo_id_rutina_opo	INT	FK -> rutinas_opo, NULL	Rutina oficial (si tipo=OPO)
usuarios_id_usuario	INT	FK -> usuarios, NOT NULL	Usuario

Diseno polimorfico: Usa tipo_rutina como discriminador. Si es OPO, se usa rutinas_opo_id; si es PERS, se usa rutinas_pers_id. El otro campo queda NULL.

3.3.14 Tabla registro_resultados

Proposito: Almacena el resultado de cada ejercicio en una sesion.

Columna	Tipo	Restricciones	Descripcion
id_resultado	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
ejercicios_id_ejercicio	INT	FK -> ejercicios, NOT NULL	Ejercicio
historial_sesiones_id_historial_sesiones	INT	FK -> historial_sesiones, NOT NULL	Sesion
valor_conseguido	DECIMAL(10,2)	NOT NULL	Valor alcanzado

3.3.15 Tabla noticias

Proposito: Noticias y actualizaciones sobre las oposiciones.

Columna	Tipo	Restricciones	Descripcion
id_noticia	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador unico
titulo	VARCHAR(500)	NOT NULL	Titulo de la noticia
contenido	TEXT	NOT NULL	Contenido completo
fecha_publicacion	DATETIME	NOT NULL	Fecha de publicacion
oposiciones_id_oposicion	INT	FK -> oposiciones, NOT NULL	Oposicion relacionada

Datos semilla: 4 noticias sobre convocatorias 2026 de PN y GC.

3.4 Relaciones entre Tablas (Foreign Keys)

Todas las claves foraneas usan la politica ON DELETE NO ACTION, ON UPDATE NO ACTION.

Tabla padre	Tabla hija	Tipo	Descripcion
oposiciones	pruebas_oficiales	1:N	Una oposicion tiene varias pruebas
oposiciones	rutinas_opo	1:N	Una oposicion tiene varias rutinas
oposiciones	usuarios	1:N	Varios usuarios para una oposicion
oposiciones	noticias	1:N	Una oposicion tiene varias noticias
pruebas_oficiales	baremos_puntuacion	1:N	Tabla de puntuacion por prueba
pruebas_oficiales	requisitos_nivel	1:N	Requisitos por nivel por prueba
pruebas_oficiales	marcas_perfil	1:N	Las marcas se asocian a pruebas
usuarios	marcas_perfil	1:N	Un usuario tiene varias marcas
usuarios	rutinas_pers	1:N	Un usuario crea varias rutinas
usuarios	historial_sesiones	1:N	Un usuario tiene varias sesiones
usuarios	settings	1:1	Cada usuario tiene una configuracion
rutinas_opo	detalle_rutina_opo	1:N	Una rutina tiene varios ejercicios
rutinas_pers	detalle_rutina_pers	1:N	Una rutina personal tiene varios ejercicios
ejercicios	detalle_rutina_opo	1:N	Un ejercicio en varias rutinas
ejercicios	detalle_rutina_pers	1:N	Un ejercicio en varias rutinas personales
ejercicios	registro_resultados	1:N	Un ejercicio con varios resultados
historial_sesiones	registro_resultados	1:N	Una sesion tiene varios resultados

\n

4. BACKEND (Node.js / Express)

4.1 Punto de Entrada: app.js

El archivo app.js es el punto de entrada del servidor. Configura Express con:

- **CORS:** Habilitado para permitir peticiones desde la app Android.
- **JSON Parser:** express.json() para parsear cuerpos de peticion JSON.
- **Static Files:** express.static('public') para servir archivos estaticos.
- **Rate Limiting:** Dos limitadores:
 - globalLimiter: 100 peticiones por 15 minutos (para todas las rutas)
 - authLimiter: 10 peticiones por 15 minutos (solo para /api/auth, previene fuerza bruta)
- **Puerto:** Configurable via variable de entorno PORT, por defecto 3000.

Rutas registradas:

Prefijo	Archivo	Rate Limiter	Descripcion
/api/auth	AuthRoute.js	authLimiter (10/15min)	Registro, login, Google Sign-In
/api/user	UsuarioRoute.js	globalLimiter	Perfil y ajustes
/api/oposiciones	OposicionRoute.js	globalLimiter	Datos de oposiciones
/api/rutinas	RutinaRoute.js	globalLimiter	Rutinas de entrenamiento
/api/rutinas-pers	RutinaPersRoute.js	globalLimiter	Rutinas personalizadas
/api/historial	ProgresoRoute.js	globalLimiter	Historial y progreso
/api/info	InfoPruebasRoute.js	globalLimiter	Info detallada de pruebas

4.2 Configuracion de Base de Datos: db.js

Crea un pool de conexiones MySQL con mysql2/promise: - host: Variable de entorno DB_HOST - user: Variable de entorno DB_USER - password: Variable de entorno DB_PASS - database: Variable de entorno DB_NAME - connectionLimit: 10 conexiones simultaneas - waitForConnections: true - queueLimit: 0 (cola ilimitada)

4.3 Middleware de Autenticacion: authMiddleware.js

Funcion: validarToken(req, res, next)

Proposito: Valida el token JWT en todas las rutas protegidas.

Flujo: 1. Extrae la cabecera Authorization de la petición 2. Valida el formato: debe ser "Bearer TOKEN" 3. Verifica la firma del token con `jwt.verify(token, JWT_SECRET)` 4. Si es válido, extrae el payload (id y email) y lo adjunta a `req.usuario` 5. Llama a `next()` para continuar al controlador

Códigos de respuesta: - 401: No hay token en la cabecera Authorization - 500: JWT_SECRET no está configurado en el servidor - 401: Token inválido o expirado

Formato del payload del token: { id: 1, email: "usuario@email.com" }

4.4 Dependencias del Backend

Paquete	Version	Propósito
bcryptjs	3.0.3	Hash y verificación de contraseñas
cors	2.8.5	Manejo de peticiones cross-origin
dotenv	17.2.3	Carga de variables de entorno
express	5.2.1	Framework web
express-rate-limit	8.3.1	Limitación de velocidad
google-auth-library	10.6.2	Verificación de tokens Google OAuth2
jsonwebtoken	9.0.3	Generación y validación de JWT
mysql2	3.16.0	Cliente MySQL con soporte Promise
jest	30.3.0	Framework de testing (dev)
nodemon	3.1.11	Recarga automática en desarrollo (dev)

4.5 Variables de Entorno (.env)

Variable	Descripción	Ejemplo
PORT	Puerto del servidor	3000
DB_HOST	Host de la base de datos	localhost
DB_USER	Usuario de MySQL	root
DB_PASS	Contraseña de MySQL	mi_contraseña
DB_NAME	Nombre de la base de datos	mydb
JWT_SECRET	Clave secreta para firmar JWT	(generada con <code>crypto.randomBytes</code>)
GOOGLE_CLIENT_ID	ID de cliente de Google OAuth	xxx.apps.googleusercontent.com

4.6 Controladores - Detalle Completo

4.6.1 AuthController.js

registrar(req, res) - POST /api/auth/registrar

Propósito: Registrar un nuevo usuario.

Body de la petición:

```
{
  "nombre": "Juan Garcia",
  "email": "juan@email.com",
  "password": "miPassword123",
  "genero": "HOMBRE",
  "peso": 80.5,
  "altura": 178,
  "oposiciones_id_oposicion": 1,
  "marcasIniciales": [
    {"id_prueba": 1, "valor": 10.5},
    {"id_prueba": 2, "valor": 12},
    {"id_prueba": 3, "valor": 210}
  ]
}
```

Proceso completo:

1. Valida que email y password estan presentes (400 si no)
2. Llama a AuthService.registrar(userData) que:
3. Obtiene una conexion exclusiva del pool
4. Inicia una TRANSACCION (BEGIN)
5. Hashea la contraseña con bcrypt (10 rondas de salt)
6. Calcula el IMC: $\text{peso} / (\text{altura}/100)^2$
7. Inserta en usuarios -> obtiene userId
8. Inserta en settings con valores por defecto (kg, km)
9. Inserta en marcas_perfil cada marca inicial
10. Hace COMMIT si todo va bien, ROLLBACK si hay error
11. Libera la conexion al pool (en finally)
12. Devuelve 201 con el userId

Respuesta exitosa (201):

```
{ "ok": true, "msg": "Usuario registrado, configurado y con marcas guardadas!", "userId": 1 }
```

Errores: 400 (falta campo), 409 (email duplicado), 500 (error servidor)

login(req, res) - POST /api/auth/login

Proposito: Autenticar un usuario con email y contraseña.

Body: { "email": "juan@email.com", "password": "miPassword123" }

Proceso: 1. Valida email y password presentes (400) 2. AuthService.login busca usuario por email en BD 3. Compara contraseña con bcrypt 4. Elimina campo password del objeto respuesta (seguridad) 5. Verifica JWT_SECRET configurado (500) 6. Genera JWT: `jwt.sign({id, email}, JWT_SECRET, {expiresIn: '24h'})` 7. Devuelve usuario + token (200)

Respuesta exitosa (200):

```
{
  "ok": true,
  "user": {
    "id_usuario": 1, "nombre": "Juan Garcia", "email": "juan@email.com",
    "genero": "HOMBRE", "peso": 80.5, "altura": 178, "imc": "25.40",
    "fecha_registro": "2026-01-15T10:30:00Z", "oposiciones_id_oposicion": 1
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

loginConGoogle(req, res) - POST /api/auth/google

Proposito: Autenticar via Google Sign-In.

Body: { "googleToken": "...", "email": "juan@gmail.com", "nombre": "Juan Garcia" }

Proceso: 1. Valida googleToken y email presentes (400) 2. AuthService.loginConGoogle: - Crea OAuth2Client con GOOGLE_CLIENT_ID - Verifica token: client.verifyIdToken({idToken, audience}) - Valida que email del token coincide - Si usuario existe -> devuelve usuario - Si es nuevo -> crea usuario con contraseña aleatoria, genero por defecto HOMBRE, peso/altura 0, settings por defecto 3. Genera JWT igual que login normal 4. Devuelve usuario + token (200)

4.6.2 UsuarioController.js

actualizarPerfil(req, res) - PUT /api/user/perfil

Proposito: Actualizar perfil fisico y marcas. **Auth:** JWT requerido.

Body:

```
{
  "userId": 1, "peso": 82.0, "altura": 178, "oposicionId": 1,
  "nuevasMarcas": [{ "id_prueba": 1, "valor": 9.8 }, { "id_prueba": 2, "valor": 14 } ]
}
```

Proceso: Valida campos (400), comprueba autorizacion userId==token.id (403), calcula IMC, actualiza usuarios, hace UPSERT en marcas_perfil, recalcula nivel con RutinaService.calcularNotaYNivel().

Respuesta: { "ok": true, "msg": "Perfil actualizado", "nuevoNivel": "INTERMEDIO", "nuevaNota": "7.50" }

actualizarSettings(req, res) - PUT /api/user/settings

Body: { "userId": 1, "unidadPeso": "kg", "unidadDistancia": "km" } **Respuesta:** { "ok": true, "msg": "Ajustes guardados" }

4.6.3 RutinaController.js

getMiEntrenamiento(req, res) - GET /api/rutinas/mi-entrenamiento/:userId/:idOposicion

Proposito: Obtener la rutina personalizada segun el nivel del usuario.

Proceso detallado: 1. Valida parametros de ruta 2. Comprueba autorizacion (403) 3. RutinaService.calcularNotaYNivel(userId, idOposicion): - Obtiene genero del usuario - Obtiene marcas del usuario para esa oposicion - Para cada marca, busca la mejor nota en baremos_puntuacion: - Si mejor_si_es_menor = 1 (tiempo): busca donde marca_usuario <= marca_baremo - Si mejor_si_es_menor = 0 (reps): busca donde marca_usuario >= marca_baremo - Calcula nota media = suma_notas /

numero_pruebas - Asigna nivel: <5 BASICO, 5-8 INTERMEDIO, >=8 AVANZADO 4.

RutinaService.obtenerRutinaCompleta(idOposicion, nivel, genero): - Busca rutinas en rutinas_opo por oposicion, nivel y genero - Para cada rutina, obtiene ejercicios de detalle_rutina_opo con JOIN a ejercicios
5. Devuelve rutina completa con nota y nivel

4.6.4 RutinaPersController.js

nuevaRutinaPersonalizada(req, res) - POST /api/rutinas-pers/crear

Proposito: Crear una rutina personalizada. **Proceso:** Verifica unicidad del nombre por usuario (transaccion), inserta rutina y ejercicios. **Respuesta (201):** { "ok": true, "msg": "Rutina creada con exito", "id": 5 }

misRutinas(req, res) - GET /api/rutinas-pers/usuario/:userId

Proposito: Listar rutinas personales con ejercicios. **Nota:** Cada ejercicio es una fila separada; el frontend agrupa por id_rutina_pers.

eliminarRutina(req, res) - DELETE /api/rutinas-pers/eliminar/:userId/:idRutina

Proposito: Eliminar rutina y ejercicios. Transaccion que verifica propiedad.

4.6.5 ProgresoController.js

guardarEntrenamiento(req, res) - POST /api/historial/registrar

Proposito: Registrar sesion de entrenamiento. **Prevencion de duplicados:** No se puede registrar la misma rutina dos veces el mismo dia.

Body:

```
{
  "userId": 1, "tipoRutina": "OP0", "idRutina": 2, "duracion": 75,
  "ejercicios": [{"id_ejercicio": 1, "valor": 12}, {"id_ejercicio": 4, "valor": 84}]
}
```

verEvolucion(req, res) - GET /api/historial/evolucion/:userId/:idEjercicio

Proposito: Obtener historial de rendimiento para graficar evolucion. **Respuesta:** Array cronologico de {fecha_entreno, valor_conseguido}.

4.6.6 OposicionController.js

- **getOposiciones** - GET /api/oposiciones/ -> Lista de oposiciones
- **getInfoOposiciones** - GET /api/oposiciones/:id -> Pruebas + noticias
- **getRequisitos** - GET /api/oposiciones/requisitos/:id/:genero -> Requisitos por nivel

4.6.7 InfoPruebasController.js

- **getInfoPruebas** - GET /api/info/:idOposicion/:genero -> Baremos detallados con tablas de puntuacion

4.7 Patron de Manejo de Errores

Todos los controladores siguen el mismo patron:

```
try {
  1. Validar campos obligatorios -> 400 Bad Request
  2. Comprobar autorizacion -> 403 Forbidden
  3. Llamar al servicio -> resultado
  4. Validar resultado -> 404 Not Found
  5. Devolver exito -> 200/201
} catch (error) {
  - console.error() para logging en servidor
  - Si es "Duplicate entry" -> 409 Conflict
  - Si no -> 500 con mensaje generico (NUNCA se expone error.message al cliente)
}
```

Respuesta exitosa: { "ok": true, "msg": "...", "data": {} } **Respuesta error:** { "ok": false, "msg": "Mensaje generico" }

4.8 Referencia Rapida de Todos los Endpoints

Sin JWT:

Metodo	Endpoint	Descripcion
POST	/api/auth/registrar	Registro de usuario (201)
POST	/api/auth/login	Login email/password (200)
POST	/api/auth/google	Login con Google (200)

Con JWT:

Metodo	Endpoint	Descripcion
PUT	/api/user/perfil	Actualizar perfil (200)
PUT	/api/user/settings	Actualizar preferencias (200)
GET	/api/oposiciones/	Listar oposiciones (200)
GET	/api/oposiciones/:id	Detalle oposicion (200)
GET	/api/oposiciones/requisitos/:id/:genero	Requisitos nivel (200)
GET	/api/info/:idOposicion/:genero	Baremos detallados (200)
GET	/api/rutinas/mi-entrenamiento/:userId/:idOposicion	Rutina personalizada (200)
POST	/api/rutinas-pers/crear	Crear rutina libre (201)
GET	/api/rutinas-pers/usuario/:userId	Listar rutinas libres (200)
DELETE	/api/rutinas-pers/eliminar/:userId/:idRutina	Eliminar rutina (200)
POST	/api/historial/registrar	Registrar entrenamiento (200)
GET	/api/historial/evolucion/:userId/:idEjercicio	Ver evolucion (200)

\n

5. FRONTEND (Android / Kotlin / Jetpack Compose)

5.1 Configuración del Proyecto

- **Namespace:** com.opofit.miapp
- **Min SDK:** 24 (Android 7.0)
- **Target SDK:** 36
- **Java Version:** 11
- **Base URL del Backend:** http://10.0.2.2:3000/ (IP del host desde el emulador Android)

Dependencias principales:

Librería	Version	Propósito
Jetpack Compose (Material 3)	BOM	UI declarativa
Firebase Auth	BOM 33.1.0	Autenticación con Firebase
Google Play Services Auth	21.2.0	Google Sign-In
Retrofit	2.9.0	Llamadas HTTP REST
Gson Converter	2.9.0	Serialización JSON
Navigation Compose	2.7.7	Navegación entre pantallas
DataStore Preferences	1.0.0	Almacenamiento local seguro
Coroutines Android	1.7.3	Programación asíncrona
Coil Compose	3.0.0-alpha06	Carga de imágenes
Material Icons Extended	1.6.0	Iconos

5.2 Arquitectura MVVM

```
VIEW (Jetpack Compose Screens)
  LoginScreen, HomeScreen, RutinasScreen...
  Observan StateFlow via collectAsState()
    |
    | Llama metodos
    v
VIEWMODEL (AndroidViewModel + StateFlow)
  AuthViewModel, RutinasViewModel, etc.
  Gestiona estado UI y llamadas API
    |
    | Usa
    v
MODEL (Data classes + API + Local Storage)
  RetrofitClient, TokenManager, DTOs
```

5.3 MainActivity

La MainActivity es la unica Activity de la aplicacion (Single Activity Pattern): 1. Crea el AuthViewModel via ViewModelProvider 2. Habilita modo edge-to-edge 3. Establece el contenido Compose con MiAppTheme 4. Crea el NavController 5. Observa authViewModel.uiState para determinar si el usuario esta logueado 6. Si esta logueado -> empieza en HOME; si no -> empieza en LOGIN

5.4 Tema Visual

Colores Modo Claro:

- Primary: #0D47A1 (Azul profundo)
- Secondary: #1565C0 (Azul medio)
- Success: #2E7D32 (Verde)
- Warning: #F57F17 (Naranja)
- Error: #C62828 (Rojo)
- Background: #FAFAFA (Gris muy claro)
- Surface: #FFFFFF (Blanco)

Colores Modo Oscuro:

- Primary: #64B5F6 (Azul claro)
- Background: #121212 (Casi negro)
- Surface: #1E1E1E (Gris oscuro)

5.5 Navegacion

Ruta	Pantalla	Parametros
login	LoginScreen	-
registro	RegisterScreen	-
home	HomeScreen	-
rutinas	RutinasScreen	-
rutinas_nivel/{nivel}	RutinasScreen	nivel: String
crear_rutina	CrearRutinaScreen	-
rutinas_libres	RutinasLibresScreen	-
detalles_rutina/{rutina_id}	DetallesRutinaScreen	rutina_id: String
entrenamientos	EntrenamientosScreen	-
registrar_entrenamiento	RegistrarEntrenamientoScreen	-
detalles_ejercicio/{ejercicio_id}	DetallesEjercicioScreen	ejercicio_id: String
perfil	PerfilScreen	-
editar_perfil	EditarPerfilScreen	-
historial	HistorialScreen	-
ajustes	AjustesScreen	-

Flujos de navegacion importantes: - **Login exitoso:** LOGIN -> HOME (limpia LOGIN del back stack) - **Logout:** HOME -> LOGIN (limpia todo el back stack) - **Finalizar entrenamiento:** ENTRENAMIENTOS -> HISTORIAL

5.6 Pantallas - Detalle Completo

5.6.1 LoginScreen (Inicio de Sesion)

Componentes: Logo con emoji, titulo "Bienvenido a OpoFit", campo email, campo contrasena con toggle de visibilidad, enlace "Olvidaste tu contrasena?", tarjeta de error, boton "Iniciar Sesion" con spinner, divisor "O continua con", boton Google Sign-In, enlace a registro.

Google Sign-In: Usa GoogleSignInOptions con DEFAULT_SIGN_IN, solicita ID Token via R.string.default_web_client_id, usa ActivityResultLauncher.

5.6.2 RegisterScreen (Registro)

Campos: Email (regex), Contrasena (6-20 chars), Confirmar contrasena, Nombre, Genero (dropdown), Peso (kg), Altura (cm), Oposicion (dropdown), Marcas iniciales opcionales, IMC auto-calculado.

5.6.3 HomeScreen (Dashboard)

TopAppBar azul con titulo "OpoFit", boton ajustes y logout. Saludo personalizado. Botones de navegacion a: Rutinas, Entrenamientos, Perfil, Historial, Ajustes.

5.6.4 RutinasScreen (Rutinas)

Carga rutina al montar. Muestra nota actual, nivel asignado, bloques de ejercicios con series/reps/descanso. Botones para iniciar entrenamiento y ver rutinas libres.

5.6.5 CrearRutinaScreen (Crear Rutina)

Nombre de rutina, filas dinamicas de ejercicios (ID, series, reps, boton eliminar), boton anadir ejercicio, boton guardar.

5.6.6 RutinasLibresScreen (Lista Rutinas Personales)

LazyColumn con tarjetas de rutinas, opcion eliminar, FloatingActionButton para crear nueva.

5.6.7 EntrenamientosScreen (Sesion Activa)

Cronometro automatico (mm:ss), lista de ejercicios con checkbox y campo de valor, boton finalizar entrenamiento. Calcula duracion y envia al backend.

5.6.8 PerfilScreen (Perfil)

Muestra datos del usuario, info de pruebas y requisitos. Boton editar perfil.

5.6.9 EditarPerfilScreen (Editar Perfil)

Campos editables de peso y altura, IMC en tiempo real, marcas por prueba, boton guardar.

5.6.10 HistorialScreen (Historial)

Selector de ejercicio, grafico de evolucion, formulario registro manual.

5.6.11 AjustesScreen (Ajustes)

Selectores de unidad peso/distancia, boton guardar, boton logout con confirmacion.

5.6.12 DetallesEjercicioScreen (Detalles Ejercicio)

Nombre, series, repeticiones, descanso, enlace a video.

5.7 ViewModels

5.7.1 AuthViewModel

Estado (AuthUiState): isLoading, error, success, isLoggedIn, userId, userEmail, userName, genero, oposicionId

Metodos: - login(email, password) - Login con credenciales - register(...) - Registro completo - loginWithGoogle(idToken) - Login con Google (Firebase + Backend) - logout() - Cierra sesion (Firebase + local) - clearError(), resetState() - Limpieza de estado

Inicializacion: En init{} llama a checkExistingSession() para restaurar sesion desde DataStore.

5.7.2 RutinasViewModel

Estado: isLoading, error, notaActual, nivelAsignado, rutinaCompleta, oposiciones

Metodos: cargarRutina(userId, oposicionId), cargarOposiciones()

5.7.3 RutinasLibresViewModel

Estado: isLoading, error, rutinas, guardadoExitoso

Metodos: cargarRutinas(userId), crearRutina(userId, nombre, ejercicios), eliminarRutina(userId, idRutina)

5.7.4 PerfilViewModel

Estado: isLoading, error, guardadoExitoso, nuevoNivel, nuevaNota, requisitos, infoPruebas

Metodos: cargarRequisitos(oposicionId, genero), cargarInfoPruebas(oposicionId, genero), actualizarPerfil(...)

5.7.5 HistorialViewModel

Estado: isLoading, error, evolucion, registradoExitoso

Metodos: cargarEvolucion(userId, idEjercicio), registrarEntrenamiento(...)

5.7.6 AjustesViewModel

Estado: isLoading, error, guardadoExitoso, unidadPeso, unidadDistancia

Metodos: guardarAjustes(userId, unidadPeso, unidadDistancia)

5.8 Capa de Red (Retrofit)

RetrofitClient (Singleton)

Configura Retrofit con Base URL desde BuildConfig.BASE_URL y conversor Gson. Instancias lazy de cada API service: rutinasApi, oposicionesApi, rutinasLibresApi, progresoApi, usuarioApi, infoPruebasApi.

Interfaces de API

AuthApi: login, register, loginWithGoogle **RutinasApi:** getMiEntrenamiento (con token en header)
RutinasLibresApi: crearRutina, getRutinasUsuario, eliminarRutina **OposicionesApi:** getOposiciones, getInfoOposicion, getRequisitos **ProgresoApi:** registrarEntrenamiento, getEvolucion **UsuarioApi:** actualizarPerfil, actualizarAjustes **InfoPruebasApi:** getInfoPruebas

Patron de autorizacion: Cada llamada protegida incluye @Header("Authorization") token: String enviado como "Bearer \$token".

BackendAuthService

Capa intermedia que envuelve las llamadas de autenticacion con el patron Result: Si response.ok -> Result.success, si no -> Result.failure con mensaje de error.

5.9 Almacenamiento Local

TokenManager (DataStore Preferences)

Almacena: auth_token (JWT), user_email, user_id, user_name. Metodos: saveToken, getToken (Flow), saveUserEmail, getUserEmail, saveUserId, getUserId, saveUserName, getUserName, clearAll.

SessionManager

Combina los Flows del TokenManager en un objeto UserSession con: token, email, userId, userName, isLoggedIn (true si token no vacio). Metodos: getCurrentSession() (Flow), saveSession(), logout().

5.10 Modelos de Datos

Request Models:

- LoginRequest(email, password)
- RegisterRequest(nombre, email, password, genero, peso, altura, oposiciones_id_oposicion, marcasIniciales)
- GoogleLoginRequest(googleToken, email, nombre)
- MarcalInicial(id_prueba, valor)

Response Models:

- AuthResponse(ok, userId, msg, message, user, token)
- UsuarioData(id_usuario, nombre, email, genero, peso, altura, imc, oposiciones_id_oposicion)
- RutinasResponse(ok, data: RutinasData)
- BloqueRutina(id_rutina_opo, bloque, ejercicios: List)
- EjercicioRutina(id_ejercicio, nombre, video_url, series, repeticiones, descanso)
- RutinaLibre(id_rutina_pers, nombre_personalizado, ejercicios_id_ejercicio, series, repeticiones)
- PuntoEvolucion(fecha_entreno, valor_conseguido)
- Oposicion(id_oposicion, nombre)
- InfoPrueba(nombre_prueba, descripcion, trucos, genero, marca_valor, nota)

5.11 Validaciones (ValidationUtils)

- isValidEmail(email): Patron `^[A-Za-z0-9+_.-]+@(.+)$`
- isValidPassword(password): 6-20 caracteres
- validateCredentials(email, password): Devuelve mensaje de error o cadena vacia

6. FLUJO DE AUTENTICACION

6.1 Login con Email y Contraseña

1. Usuario abre la app
2. MainActivity crea AuthViewModel
3. AuthViewModel.init() -> checkExistingSession()
4. TokenManager busca token guardado en DataStore
5. Si hay token -> isLoggedIn=true -> Navega a HOME
6. Si no hay token -> isLoggedIn=false -> Navega a LOGIN
7. Usuario introduce email y contraseña
8. Pulsa "Iniciar Sesión"
9. LoginScreen llama a viewModel.login(email, password)
10. AuthViewModel valida con ValidationUtils
11. Si invalido -> muestra error
12. Si valido -> llama a backendService.login()
13. BackendAuthService crea LoginRequest y llama a AuthApi
14. Retrofit serializa a JSON y envía POST /api/auth/login
15. Backend valida credenciales con bcrypt
16. Backend genera JWT con 24h de expiración
17. Backend devuelve AuthResponse (user + token)
18. AuthViewModel guarda sesión en TokenManager/SessionManager
19. Actualiza _uiState.isLoggedIn = true
20. LaunchedEffect detecta success
21. Navega a HOME (limpia LOGIN del back stack)

6.2 Login con Google

1. Usuario pulsa botón "Iniciar con Google"
2. Se lanza GoogleSignInClient con opciones: DEFAULT_SIGN_IN, requestIdToken, requestEmail
3. Google muestra selector de cuentas
4. Usuario selecciona cuenta
5. Google devuelve idToken (JWT firmado por Google)
6. LoginScreen recibe resultado via ActivityResultLauncher
7. Llama a viewModel.loginWithGoogle(idToken)
8. AuthViewModel: a. Crea credencial Firebase: GoogleAuthProvider.getCredential(idToken, null) b. Autentica en Firebase: firebaseAuth.signInWithCredential(credential).await() c. Extrae UID, email, displayName de Firebase d. Llama a backendService.loginWithGoogle(googleToken, email, name)
9. Backend verifica token con Google OAuth2Client.verifyIdToken()
10. Backend valida que email del token coincide
11. Si usuario existe -> devuelve usuario; si no -> crea nuevo

12. Backend genera JWT
13. Frontend guarda sesion
14. Navega a HOME

6.3 Persistencia de Sesion

Cuando el usuario cierra y reabre la app: 1. MainActivity.onCreate() -> crea AuthViewModel 2. AuthViewModel.init() -> checkExistingSession() 3. SessionManager.getCurrentSession() combina Flows de TokenManager 4. Si hay token guardado -> isLoggedIn = true 5. La navegacion empieza directamente en HOME

6.4 Logout

1. Usuario pulsa logout
 2. authViewModel.logout()
 3. firebaseAuth.signOut()
 4. sessionManager.logout() -> tokenManager.clearAll()
 5. DataStore Preferences se limpia
 6. _uiState se resetea
 7. Navegacion a LOGIN con popUpTo(0, inclusive=true) (limpia todo back stack)
-

7. FLUJOS DE DATOS PRINCIPALES

7.1 Calculo de Nivel del Usuario

1. Usuario actualiza sus marcas (perfil o registro)
2. RutinaService.calcularNotaYNivel(userId, idOposicion): a. Obtiene genero del usuario b. Obtiene marcas del usuario para la oposicion c. Para CADA marca:
 - Si la prueba es de tiempo (mejor_si_es_menor=1): Busca en baremos donde tiempo_usuario <= marca_baremo
 - Si la prueba es de repeticiones (mejor_si_es_menor=0): Busca en baremos donde reps_usuario >= marca_baremo
 - Obtiene la nota mas alta conseguida d. Calcula nota media = suma_notas / numero_pruebas e. Asigna nivel: <5 BASICO, 5-8 INTERMEDIO, >=8 AVANZADO
3. Se asigna la rutina correspondiente al nivel

7.2 Registro de Entrenamiento

1. Usuario completa una sesion de entrenamiento
2. La app registra duracion y valores por ejercicio
3. POST /api/historial/registrar: a. Verifica que no se haya registrado hoy para esa rutina b. Transaccion: INSERT historial_sesiones + INSERT registro_resultados por cada ejercicio c. COMMIT
4. El historial queda disponible para consultar evolucion

7.3 Crear Rutina Personalizada

1. Usuario introduce nombre y ejercicios
 2. POST /api/rutinas-pers/crear: a. Verifica nombre unico para ese usuario b. Transaccion: INSERT rutinas_pers + INSERT detalle_rutina_pers por ejercicio c. COMMIT
 3. La rutina aparece en la lista de rutinas libres
-

8. SEGURIDAD

8.1 Medidas Implementadas

Medida	Implementacion	Proteccion
Hash de contraseñas	bcryptjs con 10 rondas de salt	Contraseñas nunca en texto plano
Tokens JWT	Expiracion 24h, HMAC-SHA256	Sesiones sin estado, auth segura
Google OAuth	Verificacion server-side del token	Previene suplantacion
Rate Limiting	Global (100/15min) + Auth (10/15min)	DDoS y fuerza bruta
CORS	Habilitado	Cross-origin controlado
Autorizacion	Verificacion userId == token.id	No acceso a datos ajenos
Validacion entrada	Campos obligatorios + queries parametrizadas	Inyeccion SQL
Mensajes error	Genericos, sin detalles internos	Fuga de informacion
Connection Pool	Reutilizacion eficiente	Agotamiento de recursos
Transacciones	Todo-o-nada	Integridad de datos
Network Security	XML config solo cleartext a localhost	Solo HTTP en desarrollo

9. TESTING

9.1 Framework y Ejecucion

- **Framework:** Jest 30.3.0
- **Comando:** npm test (ejecuta jest --verbose --forceExit)
- **Ubicacion:** backend/tests/

9.2 Archivos de Test

AuthController.test.js

- Registro: 400 (falta email), 400 (falta password), 201 (exito), 409 (email duplicado), 500 (error generico)
- Login: 400 (faltan credenciales), 200 (exito con token), 401 (credenciales invalidas), 500 (JWT_SECRET no configurado)
- Login Google: 400 (falta token/email), 200 (exito), 500 (sin detalles), nombre por defecto

authMiddleware.test.js

- 401 sin Authorization header, 500 sin JWT_SECRET, 401 token invalido, next() si valido, extraccion Bearer

RutinaPersProgresoControllers.test.js

- RutinaPersController: crear (201,400,403,500), listar (200,400,403,500), eliminar (200,400,403,500)
- ProgresoController: guardar (200,400,403,500), evolucion (200,400,403,500)
- RutinaController: getMiEntrenamiento (200,400,403,404,500)
- InfoPruebasController: getInfoPruebas (200,400,404,500)

controllers.test.js

- UsuarioController: perfil (200,400,403,500), settings (200,400,403,500)
- OposicionController: listar (200,404,500), detalle (200,400,404), requisitos (200,400,404)

9.3 Cobertura

Todos los controladores y middleware cubiertos con tests que verifican: validacion entrada (400), autorizacion (403), exito (200/201), errores (500), no encontrado (404), conflictos (409).

10. DESPLIEGUE Y CONFIGURACION

10.1 Requisitos Previos

- Node.js 18+ instalado
- MySQL 5.7+ o MariaDB
- Android Studio (para el frontend)
- Cuenta de Google Cloud (para Google Sign-In)
- Cuenta de Firebase (para autenticacion)

10.2 Configuración del Backend

```
# 1. Instalar dependencias
cd backend
npm install

# 2. Crear archivo .env
cp .env.example .env
# Editar .env con los valores reales

# 3. Crear la base de datos
mysql -u root -p < ../BBDD/mydb.sql
mysql -u root -p mydb < ../BBDD/seed.sql

# 4. Iniciar en modo desarrollo
npm run dev

# 5. Iniciar en producción
npm start

# 6. Ejecutar tests
npm test
```

10.3 Configuración del Frontend

1. Abrir el proyecto frontend/ en Android Studio
2. Configurar google-services.json de Firebase
3. Configurar el default_web_client_id en strings.xml
4. Ajustar BASE_URL en build.gradle.kts si el backend no está en localhost
5. Compilar y ejecutar en emulador o dispositivo

10.4 Scripts Disponibles

Script	Comando	Descripción
Desarrollo	npm run dev	Servidor con recarga automática (nodemon)
Producción	npm start	Servidor de producción
Tests	npm test	Ejecuta todos los tests unitarios (jest)